SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to

solve problems effectively. (BL3)

Assessment Tool Weightage: 50%

QUESTIONS: 5

Duration : 1 Hour

Total Marks:50

Annexure A

Analytical Problem Solving

Code of Conduct:

I LATISHA S (192524071) (Name / Reg No) certify that this submission is my original work

and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

SOLUTIONS

1. We are given:

- One 4-gallon jug
- One 3-gallon jug
- A water pump with an unlimited supply of water
- No measuring marks on either jug

Goal: Measure exactly 2 gallons of water in the 4-gallon jug.

State Representation

Represent a state as (x, y), where:

- x = amount of water in the 4-gallon jug
- y = amount of water in the 3-gallon jug

Initial state: (0, 0)
Goal state: (2, y), where y can be any amount.

Step	Action	State (4-gallon, 3-gallon)
1	Start with both jugs empty	(0, 0)
2	Fill the 3-gallon jug completely	(0, 3)
3	Pour the 3-gallon jug into the 4-gallon jug	(3, 0)
4	Fill the 3-gallon jug completely again	(3, 3)
5	Pour water from the 3-gallon jug into the 4-gallon jug until the 4-gallon jug is full	(4, 2)
6	Empty the 4-gallon jug onto the ground	(0, 2)
7	Pour the remaining 2 gallons from the 3-gallon jug into the 4-gallon jug	(2, 0)

The final state is (2, 0). Therefore, the 4-gallon jug contains exactly 2 gallons of water.

AI Problem-Solving Interpretation

This problem can be modelled as a state-space search problem. Each pair (x, y) represents a state, and the possible operations—fill, empty, and pour—generate new states. The solution is a sequence of state transitions from the initial state (0,0) to the goal state (2,0).

Solution path:

(0,0) → (0,3) → (3,0) → (3,3) → (4,2) → (0,2) → (2,0)

Thus, exactly 2 gallons can be obtained in the 4-gallon jug using the above sequence of operations.

2. Mars Rover Agent

A Mars Rover is an intelligent agent that explores the surface of Mars, collects scientific information and samples, and sends data back to Earth. It must operate with limited communication and uncertain environmental conditions.

i. Percepts of the Agent

The percepts are the information received by the rover through its sensors and instruments. They include:

- Images and videos of the Martian surface
- Position and location of the rover
- Terrain and obstacle information
- Temperature, pressure, and atmospheric conditions
- Chemical and mineral composition of rocks and soil
- Detection of interesting rocks or samples
- Battery/power level
- Wheel movement and mechanical status
- Communication status with Earth

ii. Characterization of the Operating Environment

The Mars Rover environment can be characterized using the following properties:

Property	Characterization
Observability	Partially observable, because the rover cannot observe the entire Martian environment
Agents	Single-agent, as the rover independently performs its mission
Determinism	Stochastic, because terrain, weather, and rover conditions are uncertain
Episodic/Sequential	Sequential, because current actions affect future states
Static/Dynamic	Dynamic, because environmental conditions may change

Property	Characterization
Discrete/Continuous	Mostly continuous, as movement, time, temperature, and distance vary continuously
Known/Unknown	Partially unknown, since the rover explores previously unknown terrain

Therefore, the Mars Rover operates in a partially observable, stochastic, sequential, dynamic, continuous, and partially unknown environment.

iii. Actions the Agent Can Take

The rover can perform actions such as:

1. Move forward or backward.
2. Turn left or right.
3. Navigate to a selected location.
4. Avoid obstacles.
5. Take photographs and videos.
6. Analyze rocks and soil.

iv. Evaluation of Agent Performance

The performance of the Mars Rover can be evaluated using a performance measure. Important measures include:

- Amount and quality of scientific data collected
- Number of useful samples collected and analyzed
- Accuracy of navigation
- Distance successfully explored
- Number of scientific discoveries
- Effective use of available energy

A good agent should maximize scientific discoveries and useful data while minimizing energy consumption, risks, and equipment damage.

v. Suitable Agent Architecture

The most suitable architecture is a Model-Based Utility-Based Learning Agent, possibly with goal-based planning.

- A model-based agent maintains an internal model of the environment because Mars is only partially observable.

- A goal-based agent helps the rover achieve objectives such as reaching a target or collecting a sample.
- A utility-based agent can choose the best action by considering factors such as scientific value, energy consumption, safety, and distance.
- A learning component allows the rover to improve its decisions based on experience and newly discovered environmental information.

Conclusion: A hybrid model-based, goal-based, utility-based learning architecture is best suited for a Mars Rover because it must make autonomous decisions in an uncertain and changing environment.

3. 8-Queens Problem — Problem Formulation and Search Space

The 8-Queens problem is to place 8 queens on an 8 × 8 chessboard such that no two queens attack each other.

A queen attacks another queen if they are in the same:

- Row
- Column
- Diagonal

The objective is to find a configuration satisfying all these constraints.

Problem Components

1. Initial State: The initial state is an empty chessboard with no queens placed.

Initial state = 0 queens placed

2. State: A state represents a partial or complete placement of queens on the chessboard.

A convenient representation is:

[r1, r2, ..., rn]

where ri represents the row position of the queen in column i.

For example:

[1, 5, 8]

means:

- Queen 1 is in column 1, row 1
- Queen 2 is in column 2, row 5
- Queen 3 is in column 3, row 8

Only configurations in which no two already-placed queens attack each other are allowed.

3. Actions / Successor Function: An action consists of placing a queen in the next empty column in a row where it does not conflict with previously placed queens.

For example:

[] → [1] → [1, 5] → [1, 5, 8] → ...

A queen cannot be placed if it shares a row or diagonal with an existing queen.

4. Goal Test

The goal is reached when:

- Exactly 8 queens have been placed.
- No two queens share the same row.
- No two queens share the same column.
- No two queens share the same diagonal.

For example, one valid solution is:

[1, 5, 8, 6, 3, 7, 2, 4]

This represents one possible arrangement of 8 non-attacking queens.

5. Path Cost: Usually, the path cost is the number of queen placements.

Since every placement has equal cost:

Cost = 8

for a complete solution.

In practical search, the main objective is to find a valid solution, so path cost is less important than satisfying the constraints.

Search Space:

The search space consists of all possible arrangements generated by placing queens one column at a time.

If we place one queen in each column and allow any of 8 rows, the maximum raw search space is:

8^8 = 16,777,216 configurations

However, if we ensure that no two queens occupy the same row, the number of arrangements is:

8! = 40,320

Constraint checking further reduces the number of states explored.

Suitable Search Strategies:

1. Depth-First Search (DFS)

DFS places a queen in the first available column and continues recursively. If a conflict occurs, it backtracks and tries another position.

The process is:

Place → Check → Continue → Conflict → Backtrack → Try another position

This is the most common approach for the 8-Queens problem.

2. Backtracking Search

Backtracking is essentially a constraint-based form of DFS. It immediately rejects invalid partial configurations, making it much more efficient than searching all possible boards.

3. Heuristic Search

A heuristic such as Minimum Conflicts can be used to select positions that minimize the number of attacking pairs. This is useful for larger N-Queens problems.

Conclusion

The 8-Queens problem is a constraint satisfaction problem (CSP). It can be efficiently solved using DFS with backtracking, where invalid states are pruned as soon as conflicts are detected.

4. OLA Cab Booking — Problem-Solving Agent and Pseudocode

A customer wants to travel from a source location to a destination using a cab-booking application such as OLA. The customer can select different cab types such as Mini, Micro, Sedan, Prime, Shared, etc., based on comfort, cost, and availability.The agent must help the customer select a suitable cab and complete the booking.

Type of Problem-Solving Agent:

A suitable agent is a Goal-Based Agent with Utility-Based decision-making.

Goal-Based Agent

The primary goal is:

Transport the customer from the source location to the destination using a suitable available cab.

Utility-Based Agent

There may be several possible cabs. The agent can compare them based on:

- Fare/cost
- Comfort

- Travel time
- Distance
- Cab availability
- Customer preference
- Rating
- Estimated arrival time

Therefore, the agent can select the cab that provides the maximum utility according to the customer's preferences.

For example:

- If the customer wants low cost → select Micro or Shared.
- If the customer wants more comfort → select Sedan or Prime.
- If the customer wants fast pickup → select the cab with the shortest estimated arrival time.

Thus, the best answer is a utility-based goal-oriented problem-solving agent.

Problem Formulation:

Initial State: Customer is at the source location and has entered the destination.

Goal State: A suitable cab is successfully booked and the customer can travel to the destination.

Actions

1. Enter source location.
2. Enter destination.
3. Search for available cabs.
4. Display cab types.
5. Compare fare, ETA, and comfort.
6. Select a cab.
7. Confirm booking.
8. Assign a driver.
9. Start the trip.
10. Reach destination.

Performance Measure

The agent should maximize:

- Customer satisfaction
- Comfort
- Safety
- Availability
- Low cost
- Short waiting time
- Short travel time

Pseudocode:

```
Algorithm OLA_CAB_BOOKING_AGENT
START
Input Source
Input Destination
Input Customer_Preference
If Source or Destination is invalid then
   Display "Invalid location"
   STOP
End If
Search for available cabs
If no cab is available then
   Display "No cabs available"
   STOP
End If

For each available cab do
   Get Cab_Type
   Get Fare
   Get ETA
   Get Comfort_Level
```

Calculate Utility based on:

Customer_Preference

Fare

ETA

Comfort_Level

End For

Select cab with maximum Utility

Display selected cab details

Ask customer for confirmation

If customer confirms then

Book selected cab

Assign driver

Display driver and cab details

Start trip

Navigate from Source to Destination

If Destination is reached then

Complete trip

Calculate final fare

Display "Trip completed successfully"

End If

Else

Display "Booking cancelled"

End If

STOP

Conclusion:

The OLA cab-booking problem can be modeled as a goal-based problem-solving agent enhanced with utility-based decision-making. The agent searches available alternatives and selects the cab that best satisfies the customer's preferences while achieving the goal of reaching the destination.

5. Uniform Cost Search (UCS) — Least-Cost Path

The logistics company wants to find the least-cost path from the starting warehouse S to the destination warehouse G. Each edge in the graph has a cost representing travel expenses such as fuel and time.

We use Uniform Cost Search (UCS) to find the optimal path.

Given Graph

The directed edges and costs are:

- S → A = 1
- A → B = 3
- A → C = 1
- B → D = 3
- C → D = 1
- C → G = 2
- D → G = 3
- S → G = 12

UCS always expands the node with the lowest cumulative path cost g(n).

Step-by-Step UCS Execution:

Step	Expanded Node	Frontier / Priority Queue (Node : Cost)
1	S	A:1, G:12
2	A	C:2, B:4, G:12
3	C	D:3, G:4, B:4
4	D	G:4, B:4, G:6
5	G	Goal reached with cost 4

Explanation

1. Start at S.
    - To A: cost = 1
    - To G directly: cost = 12
2. Expand A (lowest cost = 1).

- A → B: cumulative cost = 1 + 3 = 4
- A → C: cumulative cost = 1 + 1 = 2

3. Expand C (lowest cost = 2).
   - C → D: cumulative cost = 2 + 1 = 3
   - C → G: cumulative cost = 2 + 2 = 4
4. Expand D (cost = 3).
   - D → G: cumulative cost = 3 + 3 = 6
5. The next lowest-cost goal state is G with cost 4.

Therefore, UCS terminates when it removes G from the priority queue because it has found the minimum-cost path.

Possible Paths and Costs:

Path	Total Cost
S → G	12
S → A → B → D → G	1 + 3 + 3 + 3 = 10
S → A → C → D → G	1 + 1 + 1 + 3 = 6
S → A → C → G	1 + 1 + 2 = 4

Optimal Path: $\boxed{S \rightarrow A \rightarrow C \rightarrow G}$

Minimum Total Cost: $\boxed{1 + 1 + 2 = 4}$

Final Answer

Using Uniform Cost Search, the least-cost path from warehouse S to warehouse G is:

S → A → C → G

with a minimum total travel cost of 4 units.

Why UCS is Suitable

UCS is appropriate because the routes have different costs. Unlike Breadth-First Search, which finds the path with the fewest edges, UCS considers the cumulative path cost and always expands the lowest-cost path first. With non-negative edge costs, UCS guarantees an optimal least-cost solution.